

PSG-JEPA-1D: Learned World Model vs an Exact 1-D Car Optimum

Findings from the 2026-08-16/17 iteration

Nahil Sobh*

2026-08-17

Abstract

We iterate a JEPA-style world model on a 1-D longitudinal car whose exact constant-jerk dynamics and discrete-optimum controller are known analytically. The analytical optimum stops 13.1 mm shy of the wall on a fixed 170-scenario grid; the model’s free-run rollout is measured against `W.exact_step` on 5000 random scenarios. After four rounds of experiments (grounding-only baseline, direct linear-decoder reconstruction, per-channel weighting, embedding-dimension sweep, longer training with $10\times$ data, coverage rebalance, and an MLP intrinsic decoder), the best arm halves the 10-step gap MAE from 61 mm to 33 mm, and reaches 7.5 mm to 9 mm at $k=1$ in the mid/high-velocity $\times$ low/mid-gap operating regime – under the analytical ruler. Aggregate accuracy is constrained by encoder information capacity: every loss configuration redistributes precision across $(\Delta g, v, a)$ but never grows all three at once. Coverage-based data rebalancing regressed uniformly; more data at the same epoch budget under-trained and regressed.

1 Problem setup

The world is a single car approaching a wall on a 1-D track. State is (x, v, a) ; control is jerk j ; bounds are $|a| \leq 8 \text{ m/s}^2$, $|j| \leq 20 \text{ m/s}^3$, $v \geq 0$. Discretisation is $\Delta t = 0.1 \text{ s}$.

Exact dynamics. `psgjepa1d/world.py:exact_step` subdivides each tick on events (acceleration hitting $\pm A_{\max}$, velocity hitting 0) and integrates the constant-jerk closed form on each sub-interval; verified to 7×10^{-7} vs a fine RK integrator.

Analytical optimum. `world.optimal_stop` enumerates coast lengths and terminal-brake plans that land ($v=0, a=0$) exactly. On the 170-scenario grid (initial $v_0 \in \{3, 4, \dots, 14\} \text{ m/s}$, wall $\in \{6, 7, \dots, 22\} \text{ m}$ subject to feasibility), the mean stopping excess is 13.13 mm. This is the *ruler*.

Non-dimensional scales. $T_0 = A_{\max}/J_{\max} = 0.4 \text{ s}$, $L_0 = A_{\max}^3/J_{\max}^2 = 1.28 \text{ m}$, $V_0 = A_{\max}^2/J_{\max} = 3.2 \text{ m/s}$. Physics normalisation divides (g, v, a) by $(L_0, V_0, A_{\max})$ and jerks by $J_{\max}$. The only dimensionless parameter is $\Pi = V_{\max}J_{\max}/A_{\max}^2 = 4.375$.

2 Data

$N = 2 \times 10^6$ windows of $T=7$ frames each are synthesised offline from a 7-stream stratified sampler (`data.py:gen_windows`). Each window: sample (g_0, v_0, a_0) from the mixture; then apply a piecewise-constant jerk sequence (initial jerk $\sim \text{U}(-20, 20)$; at each subsequent step, resample with probability 0.35 else hold); roll forward with `exact_step`. State storage uses gap remaining ($g_0 - x$), not absolute position. Futures are machine-zero vs the physics – verified by `tests/test_world.py`.

*Delta cluster, Illinois. Repository: <https://github.com/nahilsobh/psgjepa1d>, commit 345d71e.

The default 7-stream mixture (fractions in parens): *bulk* (32%, uniform box), *fast* (10%, $v \in [11, 14]$), *final* (8%, small gap low v), *brake* (9%, $a < -2$), *accel* (6%, $a > 2$), *bound* (13%, gap near analytical minimum brake distance), *edge* (22%, $v \approx 14$, gap $\approx 0.9\text{--}1.2\times$ brake distance).

3 Model and training

JEPA1D = encoder $\phi : R^3 \rightarrow R^D$ (MLP 3–512–512– D , GELU); residual action-conditioned predictor $\pi_\theta : R^{D+1} \rightarrow R^D$ (MLP with $z_{t+1} = z_t + \Delta$); optional linear intrinsic decoder $\psi : R^D \rightarrow R^3$ (default) or MLP $D \rightarrow h \rightarrow 3$. Grounding heads (from PSG-JEPA): static H_s predicts s_t ; transition H_d predicts multi-horizon $\Delta q_{t:t+k}$; optional velocity H_v predicts v_t .

Loss: $\mathcal{L} = \|z_{t+1} - \pi(z_t, u_t)\|^2 + \lambda_r R(\phi(S)) + \lambda_g L_{\text{ground}} + \lambda_p \|\psi(\phi(S)) - S\|^2$ where R is **sigreg** (sliced characteristic-function regulariser; $\lambda_r = 0.09$), $\lambda_g = 0.1$ (paper value), and λ_p (recon weight) is 0 or 1.0. Optimiser: AdamW, lr = 10^{-3} (the config value 3×10^{-3} diverges), weight decay 10^{-3} , batch 4096, cosine schedule over 30 epochs. Predictor target is `.detach()`-ed to avoid latent collapse.

Verify method. 5000 random ($g_0, v_0, a_0, u_{0:9}$); encode s_0 , free-run the predictor 10 steps, decode each step, compare to `exact_step` in physical units. Decoded via an MLP post-hoc decoder trained with 90%/10% split for 100 epochs on 2.4 M pairs (channel-wise val MSE reported).

4 Experiments

Each row of Table 1 is a full training run and verification. Configs identical unless noted; seed 4096 for all arms except the first two.

Table 1: Arms trained. All arms use the same encoder/predictor architecture, $\lambda_r=0.09$ sigreg, $\lambda_g=0.1$ grounding, 30 epochs on 2M windows except where noted.

Arm	Config	Purpose	Seed
baseline	no grounding	reference	3072
static / transition	one grounding term	ablation	3072
full	full grounding	PSG-JEPA-1D	3072
full.v2	same as full	seed variance	4096
full.d128	<code>wm.embed_dim=128</code>	capacity	4096
full.recon	$+\lambda_p=1$, linear decoder	direct decode loss	4096
full.recon.w	recon channel wts [1, 1, 5]	rebalance a	4096
full.d192.e60.recon	$D=192$, 60 ep, recon	capacity + longer	4096
full.recon.20M	recon, $10\times$ data, 30 ep	quantity	4096
full.recon.far	recon, mixture='far'	coverage rebalance	4096
full.recon.mlp	recon, MLP decoder $D \rightarrow 128 \rightarrow 3$	lift linear limit	4096

4.1 Aggregate results

Key observations from Table 2:

- **full.recon** halves the 10-step gap MAE ($61 \rightarrow 33$ mm) but *breaks* the acceleration channel ($41 \rightarrow 275$ mm/s²). The linear intrinsic decoder cannot fit all three channels simultaneously; the encoder trades a -precision for g -precision.
- **full.recon.mlp** preserves acceleration (28 mm/s²) but loses velocity (68 mm/s vs 24). Re-distribution again.
- **full.recon.20M**: $10\times$ more data with the same 30-epoch budget *regresses* across the board. Training loss ended at 0.003 vs 0.0005 for the 2M run – under-trained.

Table 2: Free-run rollout mean-absolute-error vs `exact_step` in physical units. $N=5000$ random scenarios, $K=10$ steps, independent-per-step jerks.

Arm	$k=1$ gap (mm)	$k=1$ v (mm/s)	$k=1$ a (mm/s ²)	$k=10$ gap (mm)	$k=10$ v (mm/s)	$k=10$ a (mm/s ²)
baseline	74	64	96	121	175	251
static	48	55	71	140	190	184
transition	141	77	77	156	148	98
full	20	29	35	51	109	114
full_v2	24	26	41	61	112	115
full_d128	19	30	37	56	93	121
full_recon	23	24	275	33	70	238
full_recon_w	28	78	117	44	143	124
full_d192_e60_recon	19	51	30	86	273	248
full_recon_20M	40	29	104	107	93	140
full_recon_far	42	45	74	90	258	158
full_recon_mlp	20	68	28	43	126	79

- `full_recon_far`: rebalancing the sampling stratification to cover the sparse $v_{lo} \times gap_{hi}$ region hurt every cell. Bad experiment design; would need much more data.
- Longer training with a bigger latent (`full_d192_e60_recon`) achieved the tightest $k=1$ decode but the predictor never recovered from a mid-training plateau and $k=10$ blew up.

4.2 Per-region gap MAE

Aggregate hides everything. Table 3 buckets samples by initial v -tercile (lo 0–4, mid 4–10, hi 10–14 m/s) and initial gap-tercile (lo 0.3–7, mid 7–15, hi 15–22 m), reporting $k=1$ / $k=10$ gap MAE in mm.

Table 3: Per-region gap MAE (mm), $k=1$ / $k=10$.

$v \backslash gap$	full_v2 (no recon)			full_recon (linear)			full_recon_mlp (MLP)		
	lo	mid	hi	lo	mid	hi	lo	mid	hi
v_{lo}	14/41	32/56	42/60	18/29	31/33	56/47	12/37	18/39	29/41
v_{mid}	14/60	23/35	36/52	9/25	17/27	36/38	13/44	20/29	24/39
v_{hi}	14/ 170	14/27	26/42	9/53	7.5/18	21/22	15/94	23/28	25/36

Best cell across any arm at $k=1$: **7.5 mm** (`full_recon`, $v_{hi} \times gap_{mid}$) – beats the 13 mm ruler by 40%. `full_recon_mlp` flattens the distribution: no cell above 29 mm at $k=1$, but no cell below 12 mm either. `full_recon_far` regressed uniformly and is omitted.

The persistent $v_{hi} \times gap_{lo} \times k=10$ blowup (170 mm on `full_v2`) is the “fast car $\rightarrow$ close wall $\rightarrow$ 10 random-jerk steps” cell: the car passes through the wall (gap goes negative) in the middle of the rollout, and no training data has negative gap. All arms degrade there; `full_recon` improves it 170 $\rightarrow$ 53 mm.

4.3 Diagnostics

Three data-related hypotheses were tested on `full_v2` and `full_recon` without retraining.

H1 (coverage). Confirmed. The ~ 20 mm aggregate is not a uniform floor. Some regions are already at 8–14 mm; others are at 40–60 mm. The heaviest contributor is $v_{lo} \times gap_{hi}$ (slow car far from wall), covered mainly by the *bulk* stream (32%) and *accel* (6%).

H2 (jerk-sequence mismatch). Rejected. Training uses 35% resample rate; verify uses independent per-step jerks. Comparing `full_v2` at $k=10$: independent jerks 61 mm, training-style jerks 63 mm. Essentially no effect. The predictor is one-step and does not depend on jerk-history correlation.

H3 (quantity). Rejected at same-epoch budget. $10\times$ data under-trained; $k=10$ gap MAE went $33 \rightarrow 107$ mm. A fair test would need ~ 100 epochs on the 20 M cache (~ 8 h GPU).

4.4 Controller closed-loop

Beyond world-model accuracy, we evaluated closed-loop stopping on the 170-scenario grid. Controller mirrors `world.optimal_stop` using the model in place of the physics: roll a coast plan forward in the model, fan trim jerks at each step, pick the plan with predicted excess closest to a target. Two policies:

Aim-tight. `target_excess = safety` – pick the smallest feasible excess. Yields low mean excess (`full` at safety 0.75 m: 2 crashes, excess 1.01 m) but heavy tail on tight margins.

Uncertainty-buffered. `target_excess = safety + kσ` with $k=1.5$ and σ = decoder gap RMSE. On `full_recon` at tight margins this actually crashed *more* because `full_recon`’s a -channel is broken – the brake-distance grid (indexed by v, a) is mis-read, so no gap-buffer helps.

Best on the 170-scenario grid. `full` at safety 0.75 m: 2 crashes, mean excess 1.01 m, i.e. $\sim 77\times$ the ruler. In the paired-common analysis on 163 scenarios all four ablation arms successfully complete, `full` averages 0.84 m excess vs 0.98 m for baseline (26% tighter p90 tail).

The $10\text{--}77\times$ excess is not fundamental to the world model; it’s the controller’s sensitivity to a -channel decoding under tight safety margins. The best world model (`full_recon`) hasn’t been evaluated with a well-designed closed-loop controller that accounts for its broken a .

5 Interpretation

5.1 Encoder capacity is the constraint

Table 4 summarises the accuracy budget for the recon variants. The encoder is 3–512–512– D with $D \in \{64, 128, 192\}$. Doubling D from 64 to 128 delivered essentially no aggregate improvement on $k=1$ gap (24 vs 19 mm, within seed variance). More striking, the linear decoder’s per-channel val MSE at $D=64$ redistributes across loss configurations without shrinking in total:

Table 4: Post-hoc MLP-decoder val MSE (100 ep, 2.16 M train), normalised. Total = sum of the three channels.

Arm	gap	v	a	total
<code>full_v2</code>	5.6e-4	1.5e-4	4e-5	7.5e-4
<code>full_recon</code>	3.5e-4	1.1e-4	8.9e-4	1.35e-3
<code>full_recon_mlp</code>	3.8e-4	5.8e-4	3e-5	9.9e-4
<code>full_recon_far</code>	1.15e-3	2.4e-4	9e-5	1.5e-3

Total budget is $\sim 10^{-3}$ across configs; the loss configuration steers *which channel* pays. A linear-decode-plus-recon loss forces gap and v tight and sacrifices a . An MLP-decode-plus-recon loss preserves gap and a and sacrifices v . Neither achieves all three.

5.2 What would push further

1. **Two-decoder training.** Force the encoder to be simultaneously linear-decodable and MLP-decodable to (g, v, a) . Directly attacks the channel-redistribution constraint.

2. **Physics-structured latent.** Reserve three latent dimensions explicitly for (g, v, a) via an alignment loss; leave the rest as free features. Trivial to implement, may bypass the encoder-capacity constraint on the three physical channels.
3. **Bigger encoder + longer training + more data, together.** $D=256$, 100+ epochs, 20 M windows. Estimated 8–10 h GPU. Brute force; will pay off only if the current 33 mm floor is not a modelling choice but a computation-budget choice.
4. **Uncertainty-aware controller with a working a .** If we recover `full_recon`’s a -channel (either via lever 1 or 2 above), the uncertainty buffer becomes usable and closed-loop excess should drop substantially.

6 Reproducibility

All code at commit 345d71e (github.com/nahilsobh/psgjepal1d). Training and evaluation are single-file:

```
python train.py --cache window_cache.npz --out ckpt/full_recon.pt \
  seed=4096 loss.recon_weight=1.0
python eval/verify_dynamics.py --arms full_v2 full_recon \
  --jerk-mode indep --report-regions
python eval/run_eval.py --arms full_recon \
  --target-excess-extra 0.036 --out eval_uncertainty.json
```

Slurm wrappers: `scripts/train_all.slurm`, `scripts/run_eval.slurm`. Environment: `/u/sobh/.conda/envs/` (torch 2.5.1+cu121, numpy 1.26.4). GPU: A100 (Delta cluster).

7 Limitations

- Verify samples (g_0, v_0, a_0) uniformly over the box but only $u_{0:9}$ independently – physically-implausible trajectories (car passing through the wall) are counted. The $v_{hi} \times gap_{lo} \times k=10$ cell in Table 3 is largely such trajectories.
- Closed-loop controller was designed for the pre-recon arms; not re-tuned for `full_recon`’s a -channel deficit.
- Every arm uses the same 30-epoch cosine schedule at $lr=10^{-3}$ except `full_d192_e60_recon`. Hyperparameter sensitivity was not swept.
- No baseline against an oracle model with $\psi = \text{identity}$ and the true predictor; that would isolate encoder loss from predictor loss.

8 Bottom line

The best learned world model (`full_recon`) achieves 7.5 mm–9 mm gap MAE at $k=1$ in the operating regime and 33 mm at $k=10$ aggregate, halving the 30-epoch baseline. The ~ 13 mm analytical ruler is a real target that the model matches or beats in-regime. Aggregate accuracy is bottlenecked by the encoder’s fixed information budget across (g, v, a) ; growing capacity requires either structural change to the loss (two-decoder or physics-structured latent) or a compute commitment (~ 8 h GPU) to fairly test the quantity lever.